TP2 noté : benchmark cryptographique sur ESP32

Écrire, vérifier, mesurer, décider

Michaël Roynard
michael.roynard@epita.fr

12 octobre 2026

TP2 : benchmark cryptographique sur ESP32 (4 h, binôme)

Décision à défendre

Vous devez choisir le profil cryptographique d'une sonde ESP32 qui envoie de petits messages de télémétrie et traite occasionnellement des blocs de 1 Kio. L'activation de ChaCha20-Poly1305 est-elle justifiée face au profil de référence, AES-256-GCM, dont les blocs AES passent par l'accélérateur matériel ?

Votre réponse doit être **mesurée, reproductible et limitée au système étudié**. Vous comparerez vitesse, flash, tas et pile. Une affirmation comme « ChaCha est plus rapide » ou « AES est meilleur », sans frontière de mesure ni provenance, ne répond pas au sujet.

Ce que vous écrivez, ce qui vous est fourni

Ce TP est d'abord un TP de code. Le projet fourni est un squelette complet qui compile, démarre sur la carte et parle le format attendu par les outils, mais dont le cœur est vide : c'est à vous de l'écrire.

Vous écrivez	Fichier	Ce qui est fourni
les statistiques d'une série	`main/statistics.c`	l'interface et ses tests hôtes
six cas cryptographiques PSA (sept avec ChaCha)	`main/crypto_cases.c`	la table des cas, SHA-256 en exemple complet, les vecteurs publiés
la fenêtre de mesure	`main/measure.c`	la création de la tâche, le contrat détaillé
l'agrégation des séries	`tools/analyze_results.py`	la lecture des captures, l'écriture du CSV, les tests
le rapport	`rapport.md` ou `rapport.pdf`	le plan attendu (section 7)

Les briques sans intérêt pédagogique pour ce cours sont fournies et ne doivent pas être modifiées : l'orchestration (`main/benchmark.c`), le format de sortie (`main/jsonl.c`), le validateur de captures, le contrôleur de profils, l'automate de mesure (`tools/run_matrix.py`), l'assembleur du rendu (`tools/make_submission.py`), le lanceur de tests (`tools/run_host_tests.sh`), les vecteurs publiés et les configurations. À la correction, seuls vos quatre fichiers sont repris de l'archive : tout le reste est remplacé par la version officielle avant de recompiler. Modifier ces briques ne peut rien rendre recevable.

Compétences évaluées

À l'issue du TP, vous saurez :

1. utiliser PSA Crypto avec une politique de clé explicite et un cycle de vie de clé sans fuite ;
2. vérifier une primitive sur un vecteur publié avant de la chronométrer ;
3. construire une fenêtre de mesure qui n'affame pas le système ;
4. comparer deux profils ESP-IDF effectivement équivalents ;
5. interpréter cycles, latence, flash, tas et pile sans confondre leurs frontières ;
6. déboguer un firmware qui plante, avec les outils d'ESP-IDF ;
7. transformer des traces machine en décision d'ingénierie, preuves à l'appui.

Prérequis et matériel

- TP1 validé sur le poste (ESP-IDF fixé par `ESP_IDF_VERSION`, accès série depuis Linux ou WSL2) ;
- **une seule carte par binôme**, ESP32 classique ou ESP32-S3. Toutes vos mesures doivent venir de cette même carte : les deux puces n'ont ni les mêmes accélérateurs ni les mêmes bus mémoire, et l'outil d'analyse refuse de mélanger des captures de cibles différentes ;
- le paquet du TP : ce sujet, le projet `benchmark/`, `tools/preflight.py` et le **guide de débogage** (`guide-debug.pdf`) ;
- aucun secret réel : les clés du banc et les vecteurs de test sont publics.

Si le diagnostic matériel ne termine pas par `READY`, signalez-le **avant** d'écrire du code. Le TP ne laisse pas le temps de réparer une installation WSL ou USB incomplète.

Architecture du projet

Le firmware

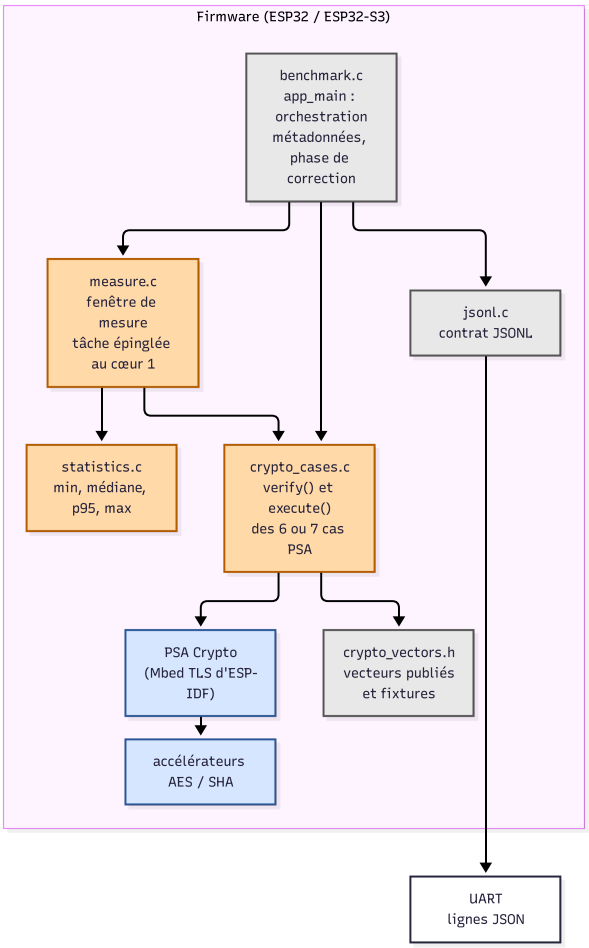


Figure 1: Architecture du firmware. En orange, ce que vous écrivez ; en gris, ce qui est fourni ; en bleu, ESP-IDF.

Le firmware suit la séparation installée au TP1 : `app_main()` compose les services et contrôle leur cycle de vie, chaque fichier possède une responsabilité et une interface minimale.

- `benchmark.c` émet les métadonnées (cible, profil, empreintes de l'ELF et du `sdkconfig`), lance la **phase de correction** (`verify()` de chaque cas), s'arrête si un cas échoue, puis lance la **phase de mesure** (`benchmark_measure_case()` pour chaque cas).
- `crypto_cases.c` expose une table de cas. Chaque cas fournit deux fonctions : `verify()`, qui prouve que la primitive est juste et qu'une entrée altérée est refusée, et `execute()`, qui réalise exactement l'opération chronométrée.
- `measure.c` exécute un cas dans une tâche dédiée, épinglée au cœur `CONFIG_CRYPTOSE_BENCHMARK_CORE` : chauffe, 32 exécutions chronométrées, statistiques, pression sur le tas et marge de pile.
- `statistics.c` réduit une série de cycles à son minimum, sa médiane, son p95 et son maximum.
- `jsonl.c` sérialise chaque résultat en une ligne JSON sur l'UART. Ce format est le **contrat** entre la carte et vos outils.

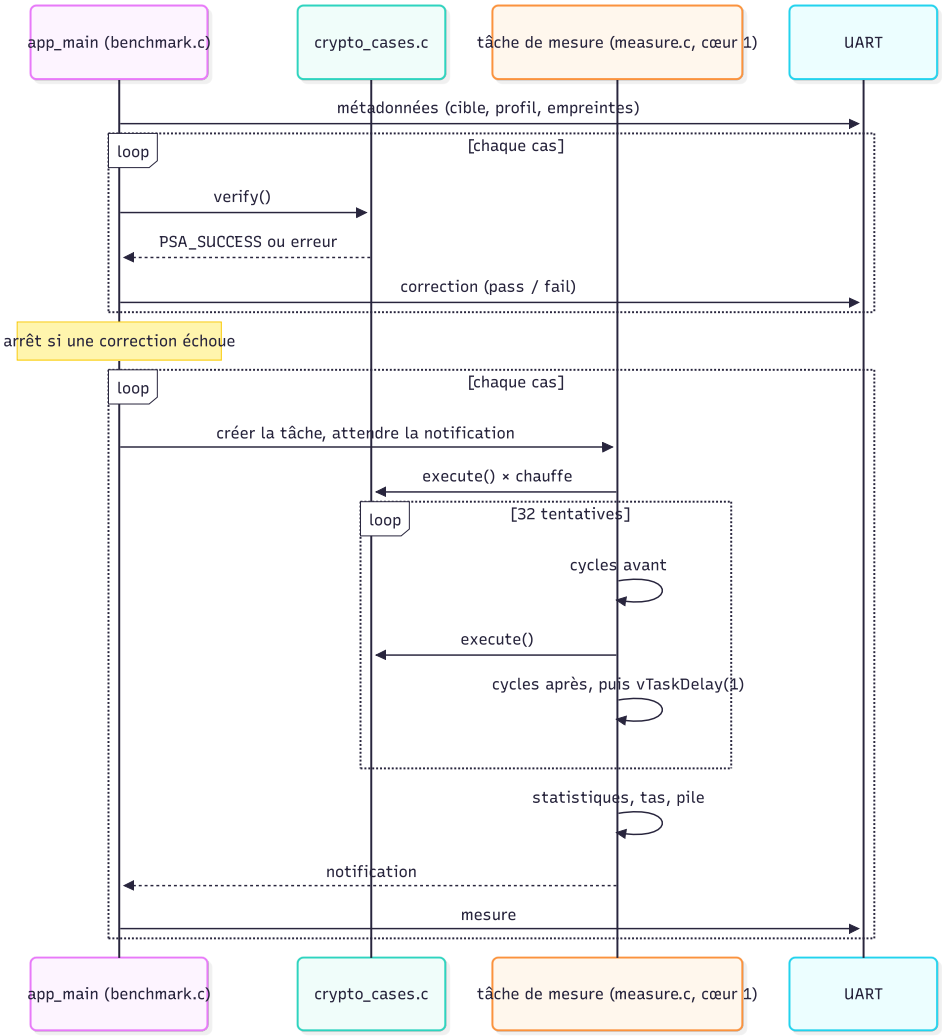


Figure 2: Déroulement d'un démarrage de la carte.

La phase de correction précède toujours la phase de mesure : chronométrer une primitive fausse ne produit que des nombres faux. Une capture n'est recevable que si **toutes** les corrections sont `pass`.

Configurations : cible, charge de travail, profil

Une compilation combine trois couches de configuration, dans cet ordre :

1. `sdkconfig.defaults` : les réglages communs (240 MHz, optimisation performance de l'application **et** de mbedtls, qui a son propre réglage, `-Os` par défaut, accélération AES et SHA, compilation reproductible, image de 4 Mio, 32 itérations) ;
2. `workloads/<charge>.sdkconfig.defaults` : la taille de la charge utile (64 ou 1024 octets) ;
3. `profiles/<profil>.sdkconfig.defaults` : `baseline`, ou `chacha20-poly1305` qui ajoute exactement quatre options.

La cible (`esp32` ou `esp32s3`) vient de `idf.py set-target`. Un ancien `sdkconfig` prime sur les valeurs par défaut : chaque combinaison est donc compilée dans un répertoire neuf.

La chaîne côté hôte

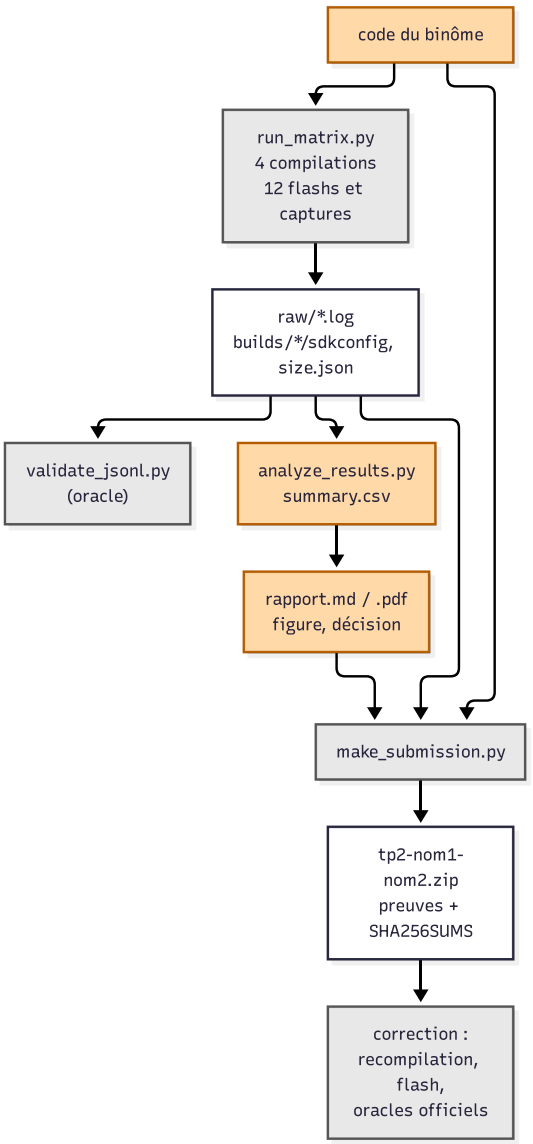


Figure 3: De votre code à la note.

`run_matrix.py` compile les quatre combinaisons, vérifie que les profils ne diffèrent que par ChaCha, puis flashe et capture trois séries par combinaison, soit douze captures, sans intervention. Chaque capture est validée au passage par `validate_jsonl.py`. Votre `analyze_results.py` agrège ensuite ces captures en `summary.csv`, sur lequel s'appuient votre figure et votre rapport. Enfin, `make_submission.py` assemble l'archive de rendu ; le correcteur la recompile, la reflashe sur une carte de l'enseignant et revalide vos captures avec les oracles officiels.

Points d'entrée de la documentation

Les en-têtes cités se trouvent dans votre installation d'ESP-IDF (`$IDF_PATH`). Ils sont la référence exacte de la version du cours.

Sujet	Où regarder
API PSA Crypto (attributs de clé, AEAD, MAC, hachage, ECDH, signature)	spécification Arm : https://arm-software.github.io/psa-api/crypto/ ; en-tête `components/mbedtls/mbedtls/tf-psa-crypto/include/psa/crypto.h`
PSA Crypto et Mbed TLS dans ESP-IDF, accélérateurs	https://docs.espressif.com/projects/esp-idf/en/stable/esp32/api-reference/protocols/mbedtls.html
Compteur de cycles	`components/esp_hw_support/include/esp_cpu.h` (`esp_cpu_get_cycle_count`)
Tas : libre, minimum, moniteur local	`components/heap/include/esp_heap_caps.h`
Tâches, notifications, marge de pile	https://docs.espressif.com/projects/esp-idf/en/stable/esp32/api-reference/system/freertos_idf.html
Générateur aléatoire matériel	https://docs.espressif.com/projects/esp-idf/en/stable/esp32/api-reference/system/random.html
Débogage (moniteur, backtrace, core dump, GDB)	`guide-debug.pdf`, fourni avec le sujet

Le modèle PSA tient en quatre étapes, identiques pour toutes les primitives à clé :

```
psa_key_attributes_t attributes = PSA_KEY_ATTRIBUTES_INIT;
psa_set_key_type(&attributes, /* type de clé */);
psa_set_key_bits(&attributes, /* taille en bits */);
psa_set_key_usage_flags(&attributes, /* usages autorisés, et eux seuls */);
psa_set_key_algorithm(&attributes, /* l'unique algorithme autorisé */);
/* 1. créer : psa_import_key(...) ou psa_generate_key(...)
   2. utiliser : psa_aead_encrypt, psa_mac_compute, psa_raw_key_agreement, psa_sign_hash...
   3. vérifier chaque psa_status_t, garder la première erreur
   4. détruire : psa_destroy_key(...) sur TOUS les chemins de sortie */
```

Une clé dont la politique autorise plus que nécessaire, ou une clé jamais détruite, est une erreur même si la mesure « marche » : la seconde fuit du tas à chaque itération, ce que montre `heap_delta_bytes`, jusqu'à `PSA_ERROR_INSUFFICIENT_MEMORY`.

Organisation : deux séances de 2 h

Séance	Section	Durée	Point de contrôle
1	0. Mise en route	15 min	diagnostic `READY`, tests hôtes lancés
1	1. Statistiques	15 min	`statistics_contract` vert
1	2. Cas cryptographiques PSA	75 min	code complet
1	3. Première exécution sur la carte	15 min	**toutes les corrections `pass`**
2	4. Fenêtre de mesure	25 min	une capture validée
2	5. Matrice automatique (lancée)	10 min	douze captures en cours
2	6. Synthèse, pendant les captures	25 min	`summary.csv`, figure
2	7. Rapport	40 min	rapport rédigé
2	8. Rendu	10 min	archive déposée

La première séance s'arrête quand la carte valide vos sept primitives. Si ce n'est pas le cas en fin de séance, notez dans votre journal ce qui échoue et pourquoi : la seconde séance reprend là. Au début de la seconde séance, rouvrez un terminal et redéfinissez les variables de la section 0.

0. Mise en route (15 min)

Depuis la racine du paquet extrait :

```
COURSE_ROOT="$(pwd)"
source "$HOME/.espressif/tools/activate_idf_v$(tr -d '[:space:]' < ESP_IDF_VERSION).sh"
PORT=/dev/ttyUSB0          # le port trouvé au TP1
python3 tools/preflight.py --require-hardware --port "$PORT" --app-dir benchmark
TARGET=esp32               # ou esp32s3 : la cible donnée par la ligne esp32 du diagnostic
SUBMISSION_ROOT="$COURSE_ROOT/submission"
cd benchmark
```

Critère de sortie : `READY`, ligne `esp32` en `PASS` (puce identifiée, flash d'au moins 4 Mio). Lancez ensuite les tests hôtes :

```
tools/run_host_tests.sh
```

Ce lanceur s'exécute dans un environnement propre : dans un terminal ESP-IDF, la chaîne d'outils de l'ESP32 masque le compilateur et le Python du système, ce qui casse les tests hôtes. Au départ, les échecs pointent uniquement vers vos zones `TODO_STUDENT` : `statistics_contract`, les tests d'agrégation et `test_student_contract.py`, qui compte les `TODO_STUDENT` restants.

1. Statistiques (15 min)

Complétez `main/statistics.c` en vous guidant par `tests/test_statistics.c`. Le contrat :

- refus d'un pointeur nul ou d'une série vide ;
- tri du tableau de travail fourni (aucune allocation) ;
- minimum et maximum ;
- médiane par rang inférieur pour un effectif pair ;
- p95 par la méthode du rang le plus proche (*nearest rank*).

Avec n valeurs triées et des rangs commençant à 1, $r_{95} = \lceil 0{,}95\, n \rceil$. Écrivez dans votre rapport la définition exacte utilisée : un « p95 » sans convention n'est pas reproductible.

2. Cas cryptographiques PSA (75 min)

Complétez `main/crypto_cases.c`. Le cas SHA-256 est entièrement écrit : lisez-le d'abord, il montre le contrat de `verify()` et de `execute()` sur la primitive la plus simple.

Cas	`execute()` chronomètre	`verify()` doit prouver
`aes-256-gcm`	import de clé, chiffrement et déchiffrement de la charge utile	le vecteur publié (texte clair + données associées) **et** le refus d'un chiffré altéré (un octet du chiffré, pas du tag)
`hmac-sha-256`	import de clé et MAC de la charge utile	le vecteur publié (RFC 4231, cas 1)
`ecdh-p256`	deux générations de clés et deux accords	les deux secrets calculés sont égaux
`x25519`	idem sur Curve25519	idem
`ecdsa-p256-sha-256`	empreinte, génération de clé, signature, vérification	la vérification **et** le refus d'une signature altérée
`chacha20-poly1305` (profil ChaCha)	comme AES-GCM	le vecteur de la RFC 8439 et le refus d'un chiffré altéré (un octet du chiffré)

Les trois cas asymétriques ne vérifient pas de vecteur publié : ce sont des contrôles d'intégration (aller-retour), plus faibles qu'un vecteur. Les RFC 5903 et 7748 donnent des scalaires et des secrets partagés publiés : vérifier ECDH et X25519 dessus est une extension possible.

Les vecteurs sont dans `main/crypto_vectors.h`, avec leur source. Le vecteur AES-256-GCM est le cas de test 16 de la spécification originale de GCM (McGrew et Viega) : il couvre à la fois un texte clair et des données associées. Un vecteur publié qui échoue désigne votre code, pas le vecteur.

Règles communes :

1. une politique de clé limitée aux usages et à l'algorithme nécessaires ;
2. la première erreur PSA est conservée et renvoyée ; un résultat faux renvoie `PSA_ERROR_INVALID_SIGNATURE` ;
3. une entrée altérée doit être refusée avec **exactement** `PSA_ERROR_INVALID_SIGNATURE` : accepter n'importe quel code d'échec masquerait une erreur d'intégration ;
4. chaque clé créée est détruite sur tous les chemins de sortie ;
5. un nonce AEAD ne doit jamais être réutilisé sous la même clé : préfixe fixe `FIXTURE_NONCE_PREFIX` suivi d'un compteur sur 32 bits. Ce compteur vit en RAM et repart de zéro à chaque démarrage, sous la même clé publique de banc : la règle n'est tenue qu'à l'intérieur d'un démarrage. Sans conséquence avec une clé de test, fatal avec une vraie clé (voir la question ci-dessous) ;
6. ne journalisez jamais clé, nonce, clair, chiffré ou tag.

Ordre conseillé : AES-GCM (le plus riche, puis réutilisable pour ChaCha), HMAC, ECDH et X25519 (une seule fonction paramétrée par la courbe), ECDSA, ChaCha.

Question pour le rapport (aléa). Pour économiser l'énergie et ne pas bruiter les mesures, la radio de la carte reste éteinte pendant tout le benchmark. Lisez la page de la documentation d'ESP-IDF sur le générateur aléatoire (section « Points d'entrée ») : quelle est alors la qualité de l'aléa utilisé par `psa_generate_key()` pour vos clés ECDH, X25519 et ECDSA, et par chaque signature ECDSA ? Est-ce acceptable ici, et que faudrait-il faire dans la sonde réelle, pour l'aléa comme pour le compteur de nonce de la règle 5 ?

3. Première exécution sur la carte (15 min)

Compilez le profil de référence, flashez et observez :

```
idf.py -B build-first set-target "$TARGET" build
idf.py -B build-first -p "$PORT" flash monitor
```

Le moniteur affiche une ligne de métadonnées puis une ligne `correctness` par cas. Les mesures sont encore `fail` : c'est attendu, la fenêtre de mesure n'existe pas encore. Quittez le moniteur avec `Ctrl+]`.

Le profil de référence ne compile pas ChaCha : son `verify()` ne s'exécute qu'avec le profil ChaCha. Compilez-le aussi, dans un répertoire et un `sdkconfig` séparés, sinon vous découvrirez une erreur de ChaCha au bout de 25 minutes d'automate (section 5). Lancez cette compilation dans un second terminal pendant que la première carte se flashe : une première compilation prend plusieurs minutes, sous WSL plus encore.

```
idf.py -B build-chacha -D SDKCONFIG=build-chacha/sdkconfig \
  -D SDKCONFIG_DEFAULTS="sdkconfig.defaults;workloads/telemetry-64.sdkconfig.defaults;profiles/chacha20-p
  set-target "$TARGET" build
idf.py -B build-chacha -p "$PORT" flash monitor
```

Critère de fin de séance : **toutes** les corrections `pass`, ChaCha compris.

Si la carte redémarre en boucle ou affiche `Guru Meditation`, ne reflashez pas au hasard : lisez la backtrace décodée par le moniteur et suivez le guide de débogage. Un `fail` sans panique se diagnostique avec `psa_status` : la valeur est un code d'erreur PSA (par exemple `-149` pour `PSA_ERROR_INVALID_SIGNATURE`) à chercher dans `psa/crypto_values.h`.

4. Fenêtre de mesure (25 min)

Complétez `measurement_task()` dans `main/measure.c`. Le contrat détaillé est en commentaire au-dessus de la fonction ; en voici la logique :

```
chauffe non mesurée (CONFIG_CRYPTOSE_BENCHMARK_WARMUP_ITERATIONS exécutions)
démarrer le moniteur du minimum du tas, lire le tas libre
pour chaque tentative (CONFIG_CRYPTOSE_BENCHMARK_ITERATIONS) :
    cycles de début
    execute()
    cycles de fin
    rendre la main à l'ordonnanceur, hors fenêtre : vTaskDelay(1)
lire le minimum du tas, arrêter le moniteur
statistiques, latences, marge de pile
```

La fenêtre ne contient que l'appel `execute()` : ni la chauffe, ni la sérialisation, ni le délai de l'ordonnanceur. Un échantillon n'est gardé que si `execute()` renvoie `PSA_SUCCESS`, et la série est invalide s'il en manque un. La soustraction non signée de deux lectures du compteur 32 bits reste juste après un unique débordement. `taskYIELD()` ne garantit pas que la tâche inactive de priorité inférieure s'exécute : sans `vTaskDelay(1)`, le chien de garde des tâches se déclenche. Ne le désactivez pas.

Le compteur de cycles compte tout ce qui s'exécute sur le cœur pendant la fenêtre, y compris les interruptions : le tick FreeRTOS à 100 Hz tombe environ 70 fois pendant un échange ECDH de 0,7 s. Une interruption ne fait qu'ajouter des cycles : le minimum (`cycles_min`) est l'estimateur le moins touché, la médiane celui qu'on agrège ici. Comparer les deux est un bon point de départ pour la partie « bruit » du rapport.

La marge de pile renvoyée par `uxTaskGetStackHighWaterMark()` est en **octets** dans ESP-IDF, contrairement au FreeRTOS standard, qui compte en mots. Le minimum du tas est global au système : c'est une observation, pas la taille intrinsèque d'une primitive.

Compilez, flashez, capturez une série dans le moniteur, puis validez-la :

```
idf.py -B build-first build && idf.py -B build-first -p "$PORT" flash monitor | tee first.log
python3 tools/validate_jsonl.py first.log
```

5. Matrice expérimentale automatique (10 min, puis sans surveillance)

Charge de travail	Charge utile	Profils
`telemetry-64`	64 octets	baseline, ChaCha
`block-1024`	1024 octets	baseline, ChaCha

Lancez l'automate, puis passez à la section 6 pendant qu'il travaille (environ 25 minutes, compilations comprises, sur un ESP32 classique) :

```
python3 tools/run_matrix.py --port "$PORT" --target "$TARGET" --output "$SUBMISSION_ROOT"
```

Il compile les quatre combinaisons dans des répertoires neufs, vérifie que les profils d'une même charge ne diffèrent que par les quatre options ChaCha, conserve `sdkconfig` et `size.json`, puis flashe et capture trois séries par combinaison :

```
submission/builds/<charge>/<baseline|chacha>/{sdkconfig,size.json}
submission/raw/<charge>/<baseline|chacha>-{01,02,03}.log
```

Chaque ligne `OK` est une capture validée. Une capture recevable contient exactement une métadonnée, six corrections et six mesures en baseline (sept avec ChaCha), uniquement des statuts `pass`, et aucun chien de garde, backtrace, panique ou redémarrage intermédiaire.

Comparaisons autorisées :

- AES-GCM, ChaCha, SHA-256 et HMAC peuvent être comparés entre charges utiles ;
- ECDH et X25519 ont `payload_bytes=0` : la charge de travail ne change pas leur frontière ;
- ECDSA inclut l'empreinte de la charge utile, mais génération, signature et vérification peuvent dominer ;
- ne comparez jamais deux algorithmes dont le champ `operation` diffère comme s'ils rendaient le même service ;
- la taille absolue de l'image n'est pas « la taille d'AES » : seul l'écart entre profils comparables peut être attribué à l'activation de ChaCha.

« Accélération AES » ne veut pas dire « AES-GCM accéléré ». L'aide de l'option `CONFIG_MBEDTLS_HARDWARE_AES` dans ESP-IDF précise : *« if the ESP32 CPU is running at 240MHz, hardware AES does not offer any speed boost over software AES »*. Gardez-le en tête en lisant vos mesures.

6. Synthèse (25 min, pendant les captures)

Complétez `summarize_series()` dans `tools/analyze_results.py` (les tests `tests/test_analyze_results.py` décrivent le contrat), puis :

```
python3 tools/analyze_results.py --raw-root "$SUBMISSION_ROOT/raw" \
  --builds-root "$SUBMISSION_ROOT/builds" --output "$SUBMISSION_ROOT/derived/summary.csv"
```

La synthèse groupe sans fusionner charge de travail, profil, algorithme et opération. Elle donne la médiane des médianes de cycles, la plage entre séries, le p95 maximal, l'écart de flash au profil de référence, le maximum de `heap_delta_bytes`, la plus petite marge de pile et le nombre de séries agrégées. Si vous calculez un débit, appelez-le **débit effectif de la frontière mesurée** : AES et ChaCha incluent ici l'import, le chiffrement et le déchiffrement.

Produisez une figure (dans `submission/derived/`) qui permet de répondre au cas de la sonde. Une figure qui place sur le même axe SHA-256 et un échange ECDH complet est trompeuse.

7. Rapport (40 min)

`rapport.md` (ou `rapport.pdf`), deux à trois pages, avec ce plan :

1. **Binôme** : les deux noms.
2. **Ce qui a été fait** : les fichiers écrits et les choix notables (politiques de clé, convention du p95, structure de la fenêtre de mesure).
3. **Ce qui marche, avec les preuves** : renvoyez aux fichiers du rendu (`evidence/`, captures validées, `summary.csv`). Une affirmation sans preuve vaut une affirmation absente.
4. **Ce qui ne marche pas, et pourquoi** : un échec diagnostiqué et expliqué rapporte plus qu'un échec caché. Dites ce que vous avez observé, ce que vous avez essayé (guide de débogage) et votre hypothèse.
5. **Décision** : pour la télémétrie de 64 octets et pour les blocs de 1 Kio, avec au moins trois valeurs chiffrées (temps, écart de flash, pire pression sur la RAM), la frontière exacte de chaque valeur et la variation entre les trois séries.
6. **Anomalies et limites** : une source de bruit ou une anomalie observée, une limite liée à votre puce, aux accélérateurs, à PSA Crypto ou au protocole, la réponse à la question sur l'aléa (section 2), et ce qu'une expérience suivante devrait isoler.

Questions pour guider l'analyse :

- l'accélération AES a-t-elle contredit votre intuition initiale ? Quelle part de la frontière AES-GCM mesurée est réellement accélérée ?
- le surcoût en flash de ChaCha est-il payé même si la primitive n'est jamais exécutée ?
- des écarts de tas de quelques dizaines d'octets se distinguent-ils du bruit global ?
- pourquoi vos temps ECDH ou X25519 ne se comparent-ils pas au hachage d'un bloc ?
- votre décision tiendrait-elle sur un microcontrôleur sans accélération AES ?

8. Rendu (10 min)

Depuis `benchmark/`, dans le terminal ESP-IDF, formatez d'abord vos trois fichiers C avec le `clang-format` fourni par ESP-IDF (celui du correcteur ; le `.clang-format` du paquet fait foi), puis produisez l'archive :

```
clang-format -i main/statistics.c main/crypto_cases.c main/measure.c
python3 tools/make_submission.py --data "$SUBMISSION_ROOT" \
  --report "$COURSE_ROOT/rapport.md" --output "$COURSE_ROOT/tp2-nom1-nom2.zip"
```

Le script copie vos sources (sans aucun répertoire de compilation), les captures, les configurations et la synthèse, relance les tests hôtes et la validation de chaque capture pour produire les preuves (`evidence/`), puis calcule `SHA256SUMS`. Rien n'y est bloquant : ce qui échoue apparaît dans les preuves et doit être expliqué dans le rapport. Déposez l'archive sur Moodle.

À la correction, vos quatre fichiers sont replacés dans le projet officiel, qui est recompilé pour les quatre combinaisons. La compilation étant reproductible, l'empreinte de l'ELF et du `sdkconfig` de chaque capture doit correspondre à cette recompilation : une capture qui ne vient pas de vos sources rendues est signalée. Vos captures sont revalidées avec les oracles officiels, les tests hôtes et des tests cachés sont rejoués, le formatage est vérifié avec `clang-format`, et une configuration inhabituelle (autre nombre d'itérations, autre taille de charge utile) détecte les constantes codées en dur. Enfin, une série est capturée sur une carte de l'enseignant et comparée à vos médianes.

Barème sur 20

Critère	Points
Cas PSA justes : vecteurs, refus des entrées altérées, politiques et destruction des clés	4
Fenêtre de mesure : chauffe, échantillons valides, ordonnanceur, tas et pile	2
Statistiques correctes	1
Code qui compile et se flashe chez le correcteur, formatage `clang-format`	2
Douze captures complètes et valides	2
Analyse des temps et des distributions	2
Analyse de la flash, du tas et de la pile	2
Effet de la charge utile, bruit, anomalies	1
Décision chiffrée adaptée au cas de la sonde	2

Critère	Points
Rapport : ce qui marche et ce qui ne marche pas, preuves, limites, question sur l'aléa	2

Un rendu qui ne compile pas chez le correcteur est plafonné à 10/20. Si une correction (`verify()`) échoue sur la carte, les mesures sont invalides : l'analyse peut encore être évaluée sur le jeu de données de secours, avec un plafond global de 15/20.